可以。先不要把项目理解成几十个Python文件，而要抓住三类数据：

$$\boxed{\text{离线参考轨迹 } q_{\text{ref}}} \quad + \quad \boxed{\text{实时测量 } y} \quad \longrightarrow \quad \boxed{\text{关节位置命令 } q_{\text{cmd}}}$$

你的完整代码链是：

```
建立机器人模型
→ 生成双脚站立初始状态
→ FDDP生成两步周期轨迹
→ continuation扩展为10步
→ 接触可行性审核
→ MuJoCo位置控制回放
→ 辨识FIR/AR预测模型
→ C0/C1/C2在线控制
→ 安全降级和故障测试
→ 硬件位置命令接口
```

一、项目里到底有哪些数据

最终轨迹文件是：

fddp_multistep_10step_ref.npz

里面最重要的变量是：

变量	维度	意义
`t_ref`	1001	0到10秒的时间
`q_ref`	1001×6	六个腿部关节参考角度
`dq_ref`	1001×6	六关节参考速度
`state_ref`	1001×25	完整浮动基座机器人状态
`tau_ref`	1000×6	FDDP优化出的分析力矩
`com_ref_world`	1001×3	质心世界坐标
`phase`	1001	当前接触阶段
`left_contact_active`	1001	左脚是否应该接触
`right_contact_active`	1001	右脚是否应该接触
`cycle_id`	1001	属于第几个两步周期

为什么`state_ref`有25维？

$$x = \begin{bmatrix} q \\ v \end{bmatrix}$$

其中：

- $q \in \mathbb{R}^{13}$：浮动基座位置和姿态7维，加六关节；
- $v \in \mathbb{R}^{12}$：浮动基座速度6维，加六关节速度。

所以：

$$\dim(x) = 13 + 12 = 25$$

二、模型代码：优化器和仿真器眼中的机器人

1. `biped_v1.urdf`

biped_v1.urdf

它主要被Pinocchio和Crocoddyl使用，提供：

- 各连杆质量和惯量；
- 关节位置；
- 关节转轴；
- 关节上下限；
- 左右脚坐标系；
- 正运动学和动力学模型。

FDDP需要机器人动力学：

$$x_{k+1} = f_k(x_k, u_k)$$

这里的f_k就是根据URDF中的质量、惯量、关节和接触信息建立的。

2. `biped_planar.xml`

biped_planar.xml

这是MuJoCo使用的模型。它额外负责：

- 地面碰撞；
- 接触力；
- 摩擦；
- 足底几何；
- 位置执行器；
- 接触传感器；
- 仿真步长。

因此两个模型的分工是：

```
URDF：优化器眼中的机器人
MJCF：物理仿真器眼中的机器人
```

3. 模型一致性检查

02_audit_urdf_mjcf_consistency.py

它检查URDF与MJCF是否具有一致的：

- 关节顺序；
- 关节轴方向；
- 关节限位；
- 连杆质量；
- 足底相对位置。

否则FDDP输出的同一个q_{ref}，在MuJoCo中可能代表不同姿态。

三、`multistep_core.py`：十步规划的规则书

multistep_core.py

这个文件故意不依赖Pinocchio、Crocoddyl或MuJoCo。它负责定义实验规则，而不是求解动力学。

1. `PhaseSpec`

```
@dataclass(frozen=True)
class PhaseSpec:
    name: str
    mode: str
    node_count: int
    swing_side: str | None
    cycle_id: int
```

它表示一个接触阶段。

例如：

```
PhaseSpec(
    name="cycle_00_right_stance_left_swing",
    mode="RIGHT_SS",
    node_count=70,
    swing_side="left",
    cycle_id=0,
)
```

含义是：

- 第0个周期；
- 右脚单支撑；
- 左脚摆动；
- 持续70个FDDP节点。

2. 接触判断

```
@property
def left_contact_active(self):
    return self.mode != "RIGHT_SS"
```

当`mode == RIGHT_SS`时，意思是右脚支撑、左脚摆动，因此左脚不接触。

类似地：

```
right_contact_active = mode != "LEFT_SS"
```

3. 一个周期怎么组成

```
DS：双脚支撑，20节点
→ RIGHT_SS：右脚支撑、左脚摆动，70节点
→ DS：双脚支撑，20节点
→ LEFT_SS：左脚支撑、右脚摆动，70节点
→ DS：双脚落地，20节点
```

一个周期节点数为：

$$20 + 70 + 20 + 70 + 20 = 200$$

因为：

$$\Delta t = 0.01\ \mathrm{s}$$

所以一个两步周期持续：

$$200 \times 0.01 = 2\ \mathrm{s}$$

五个周期就是：

$$5 \times 200 = 1000\text{个节点}$$

$$5 \times 2 = 10\ \mathrm{s}$$

因此最终得到10步、10秒轨迹。

4. `evaluate_gate`

evaluate_gate()不是优化器，而是“验收员”。

它把指标分为两类。

第一类越小越好：

$$m_i \le m_{i,\max}$$

例如：

- `solver_stop`
- `shooting_gap_norm`

- 支撑脚位置误差
- 落脚误差

第二类越大越好：

$$m_i \geq m_{i,\min}$$

例如：

- 足端净空；
- 关节限位余量；
- 前进距离；
- 轨迹持续时间。

必须全部通过，`passed`才等于`True`。

四、`05a`：生成两步周期FDDP轨迹

05a_solve_fddp_periodic_cycle.py

1. FDDP在优化什么

它建立一个有限时间最优控制问题：

$$\min_{\{x_k,u_k\}} \sum_{k=0}^{N-1} \ell_k(x_k,u_k) + \ell_N(x_N)$$

受到动力学约束：

$$x_{k+1} = f_k(x_k,u_k)$$

其中：

- x_k：第k个节点的机器人完整状态；
- u_k：六个关节的优化控制量；
- f_k：有相应接触条件的机器人动力学；
- ℓ_k：姿态、足端、控制量、接触力等运行代价；
- ℓ_N：终端状态代价。

代码里的：

```
xs
us
```

分别表示整条状态轨迹和控制轨迹。

2. 为什么FDDP允许初值暂时不可行

初始猜测不一定满足：

$$x_{k+1} = f_k(x_k,u_k)$$

两边之间的差叫shooting gap：

$$g_k = f_k(x_k, u_k) - x_{k+1}$$

FDDP可以从存在$g_k \neq 0$的轨迹开始，通过反向递推和前向rollout逐渐减少这些gap。

项目要求：

$$\|g\| \leq 10^{-7}$$

3. 平滑前进参考

代码使用五次多项式：

$$s(r) = 10r^3 - 15r^4 + 6r^5$$

然后：

$$y_{\text{ref}}(k) = y_0 + s\left(\frac{k}{N}\right)\Delta y$$

它的好处是开始和结束阶段速度、加速度比较平滑，不会突然给优化器一个位置跳变。

4. 7-stage homotopy

build_periodic_homotopy_stages()：

```
terminal_state_1
terminal_state_10
terminal_state_100
terminal_state_1000
boundary_control_1
boundary_control_10
boundary_control_30
```

前四阶段逐渐提高终端状态权重：

$$w_T : 1 \to 10 \to 100 \to 1000$$

意思是先让优化器找到大体可行的行走动作，再严格要求最终状态到达目标。

后三阶段保持：

$$w_T = 1000$$

然后逐渐提高周期边界控制权重：

$$w_B : 1 \to 10 \to 30$$

它使周期首尾的控制更加一致，方便重复连接成多步运动。

它不是普通iteration。每个stage内部都有最多500次FDDP iteration，而stage之间是不同优化问题。

5. warm start

每完成一个阶段：

```
xs = solver.xs
us = solver.us
```

下一阶段继续使用上一阶段结果：

```
solver.solve(xs, us, ...)
```

也就是：

$$(x_0^{(j+1)}, u_0^{(j+1)}) = (x_\star^{(j)}, u_\star^{(j)})$$

如果中间任何阶段没有达到：

$$\text{solver stop} \le 10^{-7}$$

$$\text{shooting gap} \le 10^{-7}$$

后面的阶段就不继续。

6. 输出限制

只有全部7阶段和所有规划gate通过，才会导出轨迹。

这里保存的：

```
q_ref = xs[:, 7:model.nq]
dq_ref = xs[:, model.nq + 6:]
tau_ref = us
```

表示：

- 去掉浮动基座位置后，提取六关节角；
- 去掉浮动基座速度后，提取六关节速度；
- 保存优化控制用于分析。

五、`05b`：2→4→6→10步continuation

05b_solve_fddp_multistep_continuation.py

1. 为什么不直接求10步

10步问题有：

- 1000个动力学节点；
- 多次接触建立和解除；
- 更多落脚约束；
- 更长的误差传播距离；
- 更差的数值条件。

所以代码固定使用：

$$2 \to 4 \to 6 \to 10$$

2. `validate_seed_metadata`

这个函数防止“名义上说是FDDP轨迹，实际上拿脚本动作当种子”。

例如，求6步时必须满足：

```
source = accepted_fddp_4step
source_step_count = 4
accepted = True
tau_ref_hardware_usage = analysis_only
```

如果元数据中出现`scripted`，代码直接拒绝。

这保证整条最终轨迹的来源可追踪：

```
两步FDDP
→ 四步FDDP
→ 六步FDDP
→ 十步FDDP
```

3. 扩展初值

`extend_control_seed()`把已经接受的控制轨迹和周期控制拼接起来。

概念上类似：

$$U^{(6)} = \begin{bmatrix} U^{(4)} \\ U^{(2)}_{\text{periodic}} \end{bmatrix}$$

但这只是新的初始猜测。拼接完成后仍然会对完整6步问题重新运行FDDP，并不是简单复制就作为最终结果。

4. 为什么10步还不是最终轨迹

`05b`输出的是：

```
fddp_multistep_10step_planning_candidate.npz
```

状态为：

```
planning_accepted_contact_pending
```

意思是：

> 优化器认为轨迹通过了规划条件，但还没有通过独立接触力审核。

因此`05b`故意不会直接创建正式`10step_ref`。

六、`05c`：检查接触在物理上是否合理

05c_report_fddp_multistep_contact_feasibility.py

核心计算位于：

contact_feasibility.py

1. 单向接触条件

地面只能推机器人，不能拉机器人，因此：

$$F_n \geq F_{n,\min}$$

其中F_n是法向力。

2. 摩擦条件

摩擦锥近似条件：

$$\|F_t\| \leq \mu F_n$$

代码使用摩擦利用率：

$$\rho_\mu = \frac{\|F_t\|}{\mu F_n}$$

项目要求：

$$\rho_\mu \leq 0.90$$

这意味着并不允许把理论摩擦能力用到100%，而是保留10%裕量。

3. CoP条件

压力中心必须位于足底支撑区域内部：

$$p_{\mathrm{CoP}} \in \mathcal{P}_{\mathrm{foot}}$$

并要求离支撑边界至少：

$$d_{\mathrm{CoP}} \geq 0.002\ \mathrm{m}$$

也就是至少2毫米裕量。

4. 滑移条件

接触脚的切向速度应很小：

$$v_{\tan} \leq 0.001\ \mathrm{m/s}$$

否则即使接触力满足摩擦锥，脚也可能已经发生滑动。

5. promotion机制

只有所有活动接触行均通过：

```
unilateral_violation = False
friction_violation = False
cop_violation = False
slip_violation = False
```

才会把候选轨迹原子化地提升为：

```
fddp_multistep_10step_ref.npz
```

并把状态改成：

```
planning_and_contact_accepted
```

这就是为什么最终轨迹比普通“solver converged”更可信。

七、`03u`：MuJoCo中的最终十步回放

03u_replay_fddp_multistep_mujoco.py

1. 严格加载

`load_multistep_reference()`会检查：

- 必须是10步；
- 必须是1001个状态样本；
- 必须是1000个控制节点；
- 时间必须严格为10毫秒网格；
- 总时间必须是10秒；
- 必须包含5个周期ID；
- 不能出现左右脚同时离地的规划节点；
- 必须经过规划和接触双重验收；
- `tau_ref`必须标记为`analysis_only`。

任何条件不满足，都拒绝运行。

2. 真正发送的是什么

回放主循环中：

```
q_command = interpolate_reference(...)
q_proxy = actuator.update(q_command)
data.ctrl[actuator_id] = q_proxy
```

所以MuJoCo接收的是关节位置：

$$q_{\mathrm{ctrl}} = q_{\mathrm{proxy}}$$

不是：

$$\tau_{\mathrm{ctrl}} = \tau_{\mathrm{ref}}$$

3. 执行器代理

执行器配置是：

- 物理仿真步长：2 ms；
- 控制周期：20 ms；
- 指令延迟：20 ms；
- 一阶时间常数：40 ms；
- 安全速度：2 rad/s；
- 每周期最大参考变化：约0.5°。

因此MuJoCo不会完美复制q_{ref}，而是模拟较慢、有延迟的真实位置执行器。

4. 每个仿真步记录什么

代码记录：

- q_{ref}
- q_{cmd}
- q_{measured}
- 关节速度
- 躯干pitch
- 躯干高度
- CoM位置
- 接触力
- 左右脚接触状态
- 接触事件
- 飞行时间
- 地面穿透深度

5. 最终验收

主要条件包括：

$$|\theta_{\mathrm{pitch}}|_{\max} \le 15^\circ$$

$$\mathrm{RMSE}_{q,\max} \le 4^\circ$$

$$\frac{h_{\min}}{h_0} \ge 0.65$$

$$\text{contact event match ratio} \ge 0.90$$

这些条件通过，证明的是：

> 已接受的FDDP轨迹可以通过有延迟的位置执行器模型在MuJoCo中执行。

不是证明真实硬件已经行走。

八、`multistep_controller.py`：最终C0/C1/C2核心

multistep_controller.py

控制器输入为：

```
ControllerInput(
    wall_time_s,
    root_pitch_rad,
    root_pitch_rate_rad_s,
    left_toe_contact,
    left_heel_contact,
    right_toe_contact,
    right_heel_contact,
)
```

也就是：

- 当前时间；
- 实际躯干pitch；
- pitch角速度；
- 四个足底接触信号。

输出为：

```
ControllerOutput(
    q_cmd,
    q_ref,
    delta_q,
    pitch_error_rad,
    alpha_rad,
    phase_offset_s,
    ...
)
```

---

1. C0：纯轨迹播放

C0不进入QP，也不进行相位同步：

$$q_{\text{cmd}} = q_{\text{ref}}$$

实际仍然经过限位器，所以更准确地说：

$$q_{\text{cmd}} = \text{Limiter}(q_{\text{ref}})$$

C0的作用是baseline：

> 如果不做主动反馈，只执行FDDP参考，机器人表现如何？

---

2. C1：固定相位FIR-QP-MPC

首先计算pitch误差：

$$e_k = \theta_k - \theta_{\mathrm{ref},k}$$

这里的θ_{ref}不是简单设为零，而是来自已接受的MuJoCo名义回放。

这是合理的，因为机器人正常行走时躯干本来就可能产生小幅周期运动。控制器应该抑制“相对名义运动的偏差”，而不是强迫pitch始终为零。

FIR模型

$$e_k \approx \sum_{i=0}^{M-1} h_i \alpha_{k-i} + d_k$$

其中：

- h_i：辨识得到的FIR系数；
- α：虚拟姿态修正输入；
- d_k：无法被输入历史解释的自然扰动部分。

AR模型

$$d_{k+1} = a_1 d_k + a_2 d_{k-1} + \cdots + a_p d_{k-p+1}$$

AR用于预测未来自然pitch误差趋势。

MPC预测

代码构造矩阵S，使得未来误差近似为：

$$E = SU + f$$

其中：

- $E = [e_{k+1}, \ldots, e_{k+N}]^{\mathsf{T}}$；
- $U = [\alpha_k, \ldots, \alpha_{k+N-1}]^{\mathsf{T}}$；
- S：由FIR系数组成的动态矩阵；
- f：历史输入和AR自由响应造成的未来误差。

预测步数：

$$N = 12$$

控制周期20 ms，所以预测时域：

$$12 \times 0.02 = 0.24\ \mathrm{s}$$

QP目标函数

大体上是：

$$J(U) = q\|SU + f\|^2 + r\|U\|^2 + s\|DU - d_{\text{prev}}\|^2$$

三部分分别代表：

- q：惩罚未来pitch误差；
- r：避免控制修正过大；
- s：避免控制修正变化太快。

正常参数：

$$q = 1500, \qquad r = 10, \qquad s = 25$$

大扰动恢复参数：

$$q = 2500, \qquad r = 5, \qquad s = 10$$

当：

$$|e_k| \geq 3^\circ$$

控制器切换到恢复参数。因为恢复模式更强调快速减小姿态误差，同时允许更积极的控制变化。

控制约束

$$|\alpha| \leq 5^\circ$$

每周期变化：

$$|\alpha_k - \alpha_{k-1}| \leq 1^\circ$$

从α映射到六关节

$$\Delta q = G\alpha$$

代码中的：

$$G = \begin{bmatrix} 0.27 & 0.062 & -0.228 & 0.27 & 0.062 & -0.228 \end{bmatrix}^{\mathsf{T}}$$

因此：

$$q_{\text{raw}} = q_{\text{ref}} + G\alpha$$

同一个α会让左右腿以相同方向进行协同的髋、膝、踝修正。

---

3. `project_alpha_to_joint_limits`

project_alpha_to_joint_limits()保证α不会让任何关节超过限位。

每个关节都必须满足：

$$q_{i,\min} \leq q_{\text{ref},i} + G_i\alpha \leq q_{i,\max}$$

由第i个关节可得到一个α允许区间：

$$\alpha \in [\alpha_{i,\min}, \alpha_{i,\max}]$$

六个关节区间取交集：

$$\alpha \in \bigcap_{i=1}^{6} [\alpha_{i,\min}, \alpha_{i,\max}]$$

最后把QP输出裁剪到交集内。

---

4. C2：C1加接触相位同步

C2会比较：

$$t_{\text{observed}} \quad 与 \quad t_{\text{expected}}$$

例如计划左脚在2.20秒落地，但实际在2.25秒落地，说明实际动作落后。

相位同步器更新：

$$\Delta t_{\text{phase}}$$

项目限制：

- 最大相位偏移：0.10 s；
- 匹配窗口：0.22 s；
- 单次最大调整：0.04 s；
- 修正增益：0.75。

参考时间变成：

$$t_{\text{motion}} = t_{\text{wall}} + \Delta t_{\text{phase}}$$

但代码没有直接把整台机器人所有参考都一起平移。它只修改与“下一个接触事件”有关的那条腿：

```
if next event is left:
    q_ref[:3] = phase_q_ref[:3]
elif next event is right:
    q_ref[3:] = phase_q_ref[3:]
```

因此：

- 左腿事件：修正左髋、左膝、左踝；
- 右腿事件：修正右髋、右膝、右踝；
- 另一条腿仍使用基础时间参考。

这是项目C2很重要的工程细节。

---

5. 四种模式的关系

模式	FIR-QP姿态修正	接触相位修正
C0	否	否
C1	是	否
phase-only	否	是
C2-final	是	是

加入`phase-only`不是为了最终部署，而是为了消融实验：

> 如果C2优于C1，究竟是相位同步产生了作用，还是其他代码产生了作用？

九、`runtime_safety.py`：控制器出错以后怎么办

runtime_safety.py

1. 状态机

```
NORMAL
→ DEGRADED
→ FALLBACK
→ SAFE_HOLD
→ FAILED
```

NORMAL

所有测量、参考和控制器输出正常。

DEGRADED

出现一般故障，例如：

- deadline miss；
- rate saturation；
- 接触事件异常。

系统仍然输出受限命令，但记录风险。

FALLBACK

C1或C2的QP、相位逻辑发生异常时，切换为C0：

$$q_{\mathrm{cmd}} \leftarrow q_{\mathrm{ref}}$$

SAFE_HOLD

测量或命令出现NaN、Inf等严重问题时，停止正常行走控制，平滑返回安全保持姿态q_{hold}。

它不是瞬间跳到保持姿态，而是经过`SafetyLimiter`：

$$q_{\text{safe},k} = q_{\text{safe},k-1} + \text{clip}\left(q_{\text{hold}} - q_{\text{safe},k-1}, -\Delta q_{\max}, \Delta q_{\max}\right)$$

FAILED

启动时缺少参考文件、文件损坏或schema错误，直接拒绝启动。

2. `SafeControllerAdapter`

它包在正常控制器外面：

```
尝试运行C2
→ C2异常：尝试C0
→ C0也异常：Safe Hold
```

所以控制器异常不会直接变成无效舵机命令。

十、最终实验代码分别证明什么

`18_validate_fddp_multistep_c2_final.py`

18_validate_fddp_multistep_c2_final.py

运行：

```
C0
C1
phase-only
C2-final
```

主要证明：

- FDDP参考本身表现怎样；
- FIR-QP是否改善pitch恢复；
- 相位同步是否改善接触时序；
- 两者合并后是否相互冲突。

`19_identify_fddp_multistep_fir.py`

19_identify_fddp_multistep_fir.py

对最终10步系统重新辨识：

$$\alpha \longrightarrow \theta_{\text{pitch}}$$

得到FIR系数h_i和AR系数a_i。

`19_tune_fddp_multistep_push_recovery.py`

19_tune_fddp_multistep_push_recovery.py

搜索不同：

$$(q, r, s)$$

组合，在恢复速度、峰值pitch、控制饱和和关节安全之间选合适参数。

`20_run_multistep_fault_injection.py`

20_run_multistep_fault_injection.py

人工注入：

- NaN/Inf；
- QP失败；
- 接触事件错误；
- 延迟；
- 饱和；
- 参考文件异常。

它主要证明安全层能否正确降级，不是证明行走性能。

`21_run_fddp_multistep_robustness_matrix.py`

21_run_fddp_multistep_robustness_matrix.py

测试：

- 前后推力；
- 不同摩擦系数；
- 斜坡；
- 台阶；
- 控制延迟；
- 传感器延迟；
- 执行器增益偏差；
- 执行器时间常数；
- 测量噪声；
- 组合扰动。

每种重要场景使用30个随机种子，并计算成功率和置信区间。

十一、哪些是最终代码，哪些是开发历史

可以这样区分：

```
01～02：站立初始状态与基础FDDP
03系列：早期站立、CoM shift和审计
04系列：单步、左右步、两步交替
05系列：最终周期和10步FDDP
06～10：C1/C2早期开发和验证
18～21：最终10步控制与鲁棒性验证
```

真正写论文方法章节时，应主要引用：

```
fddp/multistep_core.py
fddp/05a...
fddp/05b...
fddp/05c...
simulation/03u...
mpc/multistep_controller.py
mpc/runtime_safety.py
mpc/18～21
```

前面的代码主要说明开发过程和方法来源，不应该全部当作最终算法。

十二、硬件代码目前到哪里

controller_runtime.py包含：

- FIR-QP；
- C0/C1/C2类；
- 安全限位；
- 六关节顺序；
- 接触相位同步；
- 位置命令输出。

但它不包含：

- ST3215串口驱动；
- 舵机ID映射；
- 零位校准；
- IMU驱动；
- 足底传感器驱动；
- 急停；
- 实际通信超时处理。

而且这个通用runtime最初加载的是早期 `com_shift` 参考；最终10步版本通过 `mpc/multistep_controller.py` 复用了它的 `FIRQP` 和 `SafetyLimiter` 。

所以目前最准确的表述是：

> 最终十步轨迹、MuJoCo位置回放、C0/C1/C2控制算法和软件安全层已经形成；硬件侧仍需把最终多步控制器连接到实际IMU、足底接触传感器和ST3215位置通信接口。

自测一个关键问题： `05b` 求解通过以后，为什么不能立刻把10步候选轨迹当作正式参考？正确思路应该同时包含“优化器收敛”和“接触物理可行性”之间的区别。

先给结论：

- **Pinocchio**负责计算机器人的运动学和动力学，相当于“机器人数学计算器”。
- **Crocoddyl**使用Pinocchio提供的模型，建立最优控制问题并运行FDDP，相当于“轨迹优化器”。
- 它们主要出现在`fddp/`目录，不直接运行在C0/C1/C2在线控制器里。

核心关系是：

```
URDF
  ↓
Pinocchio建立机器人模型、计算动力学/运动学
  ↓
Crocoddyl建立状态、接触、代价和离散动力学
  ↓
SolverFDDP求解
  ↓
得到 x_ref、q_ref、dq_ref、tau_ref
  ↓
MuJoCo回放和C0/C1/C2使用q_ref
```

一、主要在哪些代码中使用

文件	Pinocchio	Crocoddyl	作用
04c_solve_fddp_two_step_alternating.py	是	是	建立每个接触阶段的基础action model
05a_solve_fddp_periodic_cycle.py	是	是	求解两步周期FDDP和7-stage homotopy
05b_solve_fddp_multistep_continuation.py	是	是	求解4、6、10步FDDP
05c_report_fddp_multistep_contact_feasibility.py	是	间接使用	重新计算FDDP轨迹的接触力和可行性
`03系列`	是	是	早期站立、CoM shift和homotopy开发
`01/02系列`	主要是Pinocchio	部分使用Crocoddyl	初始状态与站立问题

最重要的是：

```
04c：基础积木
05a：两步周期问题
05b：最终多步问题
```

`05a`和`05b`会复用`04c`中已有的action model构建方法，所以不是全部从头重写。

二、Pinocchio具体起什么作用

项目中通常这样导入：

```
import pinocchio as pin
```

1. 从URDF建立机器人动力学模型

在`05a`中：

```
model = pin.buildModelFromUrdf(
    str(URDF_PATH),
    pin.JointModelFreeFlyer()
)
```

这里做了两件事。

第一，从URDF读取：

- 连杆质量；
- 转动惯量；
- 关节连接；
- 关节轴；
- 关节限位；
- 各坐标系的位置。

第二，加入浮动基座：

```
pin.JointModelFreeFlyer()
```

因为双足机器人躯干不是固定在世界坐标系上的。它可以：

- 平移；
- 俯仰；
- 侧倾；
- 转向。

最终配置变量为：

$$q \in \mathbb{R}^{13}$$

包括：

- 浮动基座位置和四元数：7维；
- 六个腿部关节：6维。

速度为：

$$v \in \mathbb{R}^{12}$$

包括：

- 浮动基座线速度与角速度：6维；
- 六关节速度：6维。

所以完整状态：

$$x = \begin{bmatrix} q \\ v \end{bmatrix} \in \mathbb{R}^{25}$$

---

2. 计算正运动学

例如：

```
pin.forwardKinematics(model, data, q)
pin.updateFramePlacements(model, data)
```

正运动学解决的问题是：

> 已知基座姿态和六个关节角，左右脚现在在哪里？

数学上可以写成：

$$^{W}T_{\text{foot}} = FK_{\text{foot}}(q)$$

输出是足端相对于世界坐标系的位姿：

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \in SE(3)$$

其中：

- $R \in SO(3)$：足端旋转；
- $p \in \mathbb{R}^3$：足端位置。

项目用它检查：

- 支撑脚有没有移动；
- 摆动脚有没有抬起来；
- 落脚位置是否准确；
- 周期首尾足端位置是否一致。

---

3. 使用 `SE3` 表示足端位姿

例如：

```
pin.SE3(rotation, translation)
```

在你的项目中，左右脚参考位姿不是单纯一个位置，而是：

$$T_{\text{foot,ref}} = (R_{\text{ref}}, p_{\text{ref}})$$

这样既能要求足端到达目标位置，也能要求脚底保持正确方向。

项目还会建立足底接触坐标系：

```
sole = pin.Frame(
    ...,
    pin.SE3(np.eye(3), sole_offset),
    pin.FrameType.OP_FRAME,
)
```

这是因为URDF中的ankle frame不一定正好位于真实足底平面。代码向下偏移约43 mm，建立`sole contact frame`。

---

4. 计算质心

例如：

```
com = pin.centerOfMass(model, data, q)
```

得到：

$$c(q) = \begin{bmatrix} c_x \\ c_y \\ c_z \end{bmatrix}$$

FDDP需要控制质心，是因为行走不能只规定“脚怎么动”，还要保证机器人整体质量没有跑出支撑区域。

在项目中，Pinocchio计算实际质心，Crocoddyl再构造质心误差：

$$e_{\mathrm{CoM}} = c(q) - c_{\mathrm{ref}}$$

然后把它放进优化目标。

---

5. 计算姿态误差

代码中有：

```
pin.log3(reference.rotation.T @ actual.rotation)
```

旋转矩阵不能直接普通相减，所以先计算相对旋转：

$$R_{\mathrm{err}} = R_{\mathrm{ref}}^{\mathsf{T}} R_{\mathrm{actual}}$$

再使用对数映射：

$$\phi_{\mathrm{err}} = \log(R_{\mathrm{err}}) \in \mathbb{R}^3$$

它把旋转误差转换成一个三维旋转向量。

其模长：

$$\theta_{\mathrm{err}} = \left\| \log\left(R_{\mathrm{ref}}^{\mathsf{T}} R_{\mathrm{actual}}\right) \right\|$$

就是旋转误差角。

项目用它检查：

- 支撑脚旋转误差；
- 落脚旋转误差；
- 周期首尾基座姿态误差。

---

6. 计算多体动力学

虽然很多底层调用被Crocoddyl封装了，但实际动力学仍由Pinocchio计算。

浮动基座机器人的动力学可以写成：

$$M(q)\dot{v} + h(q,v) = S^{\mathsf{T}}\tau + J_c(q)^{\mathsf{T}}\lambda$$

其中：

- $M(q)$：质量矩阵；
- $h(q,v)$：重力、科氏力和离心力；
- S：选择六个可驱动关节；
- τ：六关节控制力矩；
- $J_c(q)$：接触点Jacobian；
- λ：地面对脚的接触力。

Pinocchio负责计算：

- $M(q)$
- $h(q,v)$
- $J_c(q)$
- 足端速度；
- 动力学及其导数。

Crocoddyl在此基础上建立接触动力学。

三、Crocoddyl具体起什么作用

项目中：

```
import crocoddyl
```

如果说Pinocchio回答：

> 给定q, v, τ，机器人会怎样运动？

那么Crocoddyl回答：

> 为了让机器人按要求走路，每个时刻应该选择什么τ和状态轨迹？

1. 定义机器人状态空间

在`05a`和`05b`中：

```
state = crocoddyl.StateMultibody(model)
```

这里把Pinocchio模型包装为Crocoddyl能使用的状态：

$$x = (q, v)$$

为什么不用普通向量加减?

因为q中包含四元数，姿态不属于普通欧氏空间。Crocoddyl的`StateMultibody`知道应该怎样：

- 比较两个机器人状态；
- 对状态施加增量；
- 处理四元数；
- 计算状态差及其导数。

例如状态误差不是简单：

$$q - q_{\text{ref}}$$

而更接近：

$$\delta x = x \ominus x_{\text{ref}}$$

其中$\ominus$表示流形上的状态差。

---

2. 定义浮动基座驱动方式

```
actuation = crocoddyl.ActuationModelFloatingBase(state)
```

机器人有12维广义速度，但只有6个关节电机。

因此：

$$u = \tau \in \mathbb{R}^6$$

基座的六个自由度不能直接施加电机力矩，只能通过：

- 腿部关节；
- 地面接触力；

间接改变基座运动。

`ActuationModelFloatingBase`正是用来表达这种欠驱动系统。

---

3. 定义支撑脚接触

基础两步代码中使用：

```
contacts = crocoddyl.ContactModelMultiple(state, nu)
```

再根据接触阶段加入：

```
crocoddyl.ContactModel6D(...)
```

例如右脚支撑、左脚摆动时：

```
右脚：加入6D接触模型
左脚：不加入接触，改为跟踪摆动轨迹
```

六维接触大致要求支撑脚：

$$v_{\text{foot}} = 0$$

并通过接触约束求出保持足底不动所需的接触力λ。

不同阶段对应不同动力学：

```
DS：左脚接触 + 右脚接触
RIGHT_SS：只有右脚接触
LEFT_SS：只有左脚接触
```

这也是为什么每个时间节点可能有不同的action model。

---

4. 定义优化代价

Crocoddyl通过“残差 + 代价”表达机器人应该做什么。

状态跟踪

```
state_residual =
    crocoddyl.ResidualModelState(state, x_reference, nu)
```

残差：

$$r_x = x \ominus x_{\text{ref}}$$

代价：

$$\ell_x = \frac{1}{2} r_x^{\mathsf{T}} W_x r_x$$

作用是让机器人状态接近参考姿态。

控制代价

```
control_residual =
    crocoddyl.ResidualModelControl(state, np.zeros(nu))
```

残差：

$$r_u = u - u_{\text{ref}}$$

这里通常：

$$u_{\text{ref}} = 0$$

代价：

$$\ell_u = \frac{1}{2} u^{\mathsf{T}} W_u u$$

它不是要求所有力矩都等于零，而是防止优化器使用不必要的巨大力矩。

质心跟踪

```
ResidualModelCoMPosition(state, com_target, nu)
```

残差：

$$r_c = c(q) - c_{\text{target}}$$

使质心按照规划方向平滑移动。

足端位姿跟踪

```
ResidualModelFramePlacement(...)
```

对摆动脚，它要求：

$$T_{\text{swing}}(q) \approx T_{\text{swing,ref}}$$

这样形成：

- 抬脚；
- 向前摆；
- 落脚。

关节限位

项目使用：

```
ActivationModelQuadraticBarrier(
    ActivationBounds(lower, upper)
)
```

当关节位于安全区间内时，代价很小或为零；接近或超过限位时，代价迅速增大。

摩擦和CoP代价

最终 `05a/05b` 还添加了自定义：

```
ResidualModelPlanarSupportCone
```

再配合：

```
ActivationModelQuadraticBarrier
```

用于惩罚：

- 法向力错误；
- 摩擦锥越界；
- CoP接近或超过足底边界。

---

5. 建立接触动力学

基础代码使用：

```
crocoddyl.DifferentialActionModelContactFwdDynamics(...)
```

它将以下内容组合起来：

```
Pinocchio机器人动力学
+ 当前支撑脚接触
+ 六关节驱动
+ 当前节点的所有代价
```

从而得到连续时间动力学：

$$\dot{x} = f(x, u)$$

在接触约束下，需要联合求解：

$$M(q)\dot{v} + h(q, v) = S^{\mathsf{T}}\tau + J_c^{\mathsf{T}}\lambda$$

以及接触加速度条件，例如：

$$J_c\dot{v} + \dot{J}_c v = 0$$

由此求出：

- 机器人加速度$\dot{v}$；
- 接触力λ。

---

6. 离散化动力学

项目使用：

```
crocoddyl.IntegratedActionModelEuler(
    differential,
    dt
)
```

将连续动力学：

$$\dot{x} = f(x, u)$$

离散为：

$$x_{k+1} = f_k(x_k, u_k)$$

直观近似为：

$$x_{k+1} \approx x_k + \Delta t\, \dot{x}_k$$

项目中：

$$\Delta t = 0.01\ \mathrm{s}$$

每个FDDP节点都有一个这样的离散action model。

---

7. 建立完整Shooting Problem

在`05a`中：

```
problem = crocoddyl.ShootingProblem(
    x0,
    running_models,
    terminal_model
)
```

它包含：

- `x0`：机器人初始状态；
- `running_models`：前200或1000个节点的动力学和代价；
- `terminal_model`：最后一个节点的终端代价。

对应数学问题：

$$\min_{\{x_k, u_k\}} \sum_{k=0}^{N-1} \ell_k(x_k, u_k) + \ell_N(x_N)$$

满足：

$$x_0 = x_{\text{initial}}$$

$$x_{k+1} = f_k(x_k, u_k)$$

由于不同节点的接触模式不同，f_k也会变化。

---

8. 使用FDDP求解

在`05a`中：

```
solver = crocoddyl.SolverFDDP(problem)
```

然后：

```
solver.solve(
    xs_initial,
    us_initial,
    max_iterations,
    False,
)
```

输入：

- `xs_initial`：初始状态轨迹猜测；
- `us_initial`：初始控制轨迹猜测；
- `max_iterations`：最多迭代次数。

FDDP每次迭代主要包括：

```
根据当前轨迹计算动力学、代价和导数
→ backward pass
→ 得到前馈控制修正和反馈增益
→ forward rollout / line search
→ 更新xs、us
→ 检查cost和shooting gap
```

输出：

```
solver.xs
solver.us
```

即：

$$X^\star = \{x_0^\star, \dots, x_N^\star\}$$

$$U^\star = \{u_0^\star, \dots, u_{N-1}^\star\}$$

四、在`05a`里二者如何合作

`05a`的核心流程可以直接对应为：

第一步：Pinocchio加载模型

```
model = pin.buildModelFromUrdf(...)
```

得到机器人动力学模型。

第二步：Pinocchio计算初始信息

```
pin.centerOfMass(...)
forwardKinematics(...)
updateFramePlacements(...)
```

得到：

- 初始CoM；
- 左右脚起始位姿；
- 足底坐标系。

第三步：Crocoddyl建立状态和驱动

```
state = crocoddyl.StateMultibody(model)
actuation = crocoddyl.ActuationModelFloatingBase(state)
```

第四步：为200个节点分别建立模型

每个节点根据DS、LEFT_SS、RIGHT_SS决定：

- 哪只脚有接触；
- 哪只脚跟踪摆动轨迹；
- 当前CoM目标；
- 当前状态目标；
- 当前足端目标；
- 当前摩擦和CoP代价。

第五步：组成Shooting Problem

```
problem = crocoddyl.ShootingProblem(...)
```

第六步：FDDP求解

```
solver = crocoddyl.SolverFDDP(problem)
solver.solve(...)
```

第七步：Pinocchio重新检查结果

求解后代码再次使用：

```
pin.forwardKinematics(...)
pin.centerOfMass(...)
pin.log3(...)
```

计算：

- CoM轨迹；
- 支撑脚误差；
- 落脚误差；
- 周期首尾误差。

所以Pinocchio不仅参与求解前建模，也参与求解后的结果审计。

五、在`05b`里二者如何合作

05b_solve_fddp_multistep_continuation.py与`05a`基本相同，但把两步周期复制成多个周期。

例如10步：

$$N = 1000$$

代码为每个节点建立对应接触模型：

```
for node in range(horizon_nodes):
    action_model = base._build_action_model(...)
    running_models.append(action_model)
```

然后：

```
problem = crocoddyl.ShootingProblem(
    x0,
    running_models,
    terminal_model
)
```

最终：

```
solver = crocoddyl.SolverFDDP(problem)
```

区别在于它会分阶段求解：

```
4步 → 6步 → 10步
```

Pinocchio提供的机器人模型没有变化，变化的是Crocoddyl最优控制问题的时间长度、参考轨迹和节点数量。

六、以“右脚支撑、左脚摆动”节点为例

假设某一节点是：

```
RIGHT_SS
```

也就是右脚支撑、左脚摆动。

Pinocchio负责

根据当前q, v计算：

- 右脚位置和姿态；
- 左脚位置和姿态；
- 左右脚Jacobian；
- CoM位置；
- 质量矩阵$M(q)$；
- 非线性项$h(q, v)$。

Crocoddyl负责

为该节点设置：

- 右脚6D接触；
- 左脚摆动位姿跟踪；
- CoM跟踪；
- 状态跟踪；
- 控制量惩罚；
- 关节限位；
- 支撑力摩擦/CoP代价。

然后FDDP决定：

$$\tau_k^\star$$

使得下一时刻的机器人状态尽可能接近计划状态，同时右脚不滑、左脚按照摆动轨迹运动。

七、它们是否用于在线C0/C1/C2

最终在线控制器：

multistep_controller.py

没有直接导入：

```
pinocchio
crocoddyl
```

因为它不再在线求解完整机器人动力学问题，而是直接读取离线结果：

$$q_{\mathrm{ref}}(t)$$

然后计算：

$$q_{\mathrm{cmd}} = q_{\mathrm{ref}} + G\alpha$$

因此分工为：

阶段	使用的软件
离线轨迹优化	Pinocchio + Crocoddyl
物理回放验证	MuJoCo
在线姿态控制	FIR-QP-MPC
接触相位同步	C2 phase synchronizer
硬件命令	六关节位置命令

这也说明你的MPC不是把Crocoddyl FDDP放在每个20 ms周期内重新求解，而是在离线FDDP参考上进行低维快速反馈修正。

八、最容易混淆的一点

Crocoddyl输出的：

$$u_k^\star = \tau_{\mathrm{ref},k}$$

在当前项目中没有直接发送到MuJoCo或硬件。

真正执行的是：

$$q_{\mathrm{cmd}} = q_{\mathrm{ref}} + \Delta q$$

因此：

- Pinocchio和Crocoddyl负责“设计名义动作”；
- MuJoCo负责“检查位置执行器能否执行动作”；
- C1/C2负责“实际执行偏离名义动作时怎样修正”。

一句话概括：

> Pinocchio告诉Crocoddyl机器人在物理上会怎样运动；Crocoddyl利用这些信息，通过FDDP寻找一条满足接触、姿态、质心和关节约束的最优轨迹。